DuckLake v1.0: The Lakehouse Format Built on SQL Reaches Production-Readiness

The DuckDB team
2026-04-13 | 23 min

TL;DR: We are happy to release DuckLake v1.0, a production-ready lakehouse format specification built on SQL. Its reference implementation, the `ducklake` DuckDB extension, is available as of today in DuckDB v1.5.2.

In May 2025, we published the DuckLake manifesto, where we explained what motivated us to work on DuckLake. Here's a quick recap: we basically outlined how, in our view, it makes much more sense to store all metadata of a lakehouse in a **database** rather than in scattered files in object storage. This is why we created DuckLake.

> The manifesto is much more compelling, we recommend you read it!

Today, we are happy to announce **DuckLake v1.0**, almost a year after we released our first sketch of the specification. This is a production-ready release with guaranteed backward-compatibility. DuckLake v1.0 ships a stable specification, a feature-rich and fast reference implementation (the DuckDB `ducklake` extension), as well as a roadmap for future development.

DuckLake at a Glance

If you're new here, you may be wondering: what is DuckLake? DuckLake is a lakehouse format, which allows you to store your data on object storage and access it as a database, similarly to Delta Lake with Unity Catalog and Iceberg with Lakekeeper.

A key difference between other formats and DuckLake is that DuckLake stores all metadata in a database, which is commonly referred to as the catalog. This database can be any system that speaks SQL, supports primary keys and is able to persist data in tables. The DuckLake specification defines the metadata tables needed for a specific version along with the supported data types and a reference on how to retrieve table metadata to perform operations on the lakehouse.

We also developed an implementation of the specification using DuckDB as the engine, the DuckDB `ducklake` extension. This extension implements all of the specification and supports three main catalogs: SQLite, PostgreSQL and DuckDB (yes, DuckDB can be a catalog too).

Over the last year, we kept working on the DuckLake specification internally and with the community. We enabled adding existing Parquet files without creating a deep copy, introduced Iceberg compatibility, and rolled out support for the geometry and the variant types. We also released several guides and migration scripts from DuckDB to DuckLake.

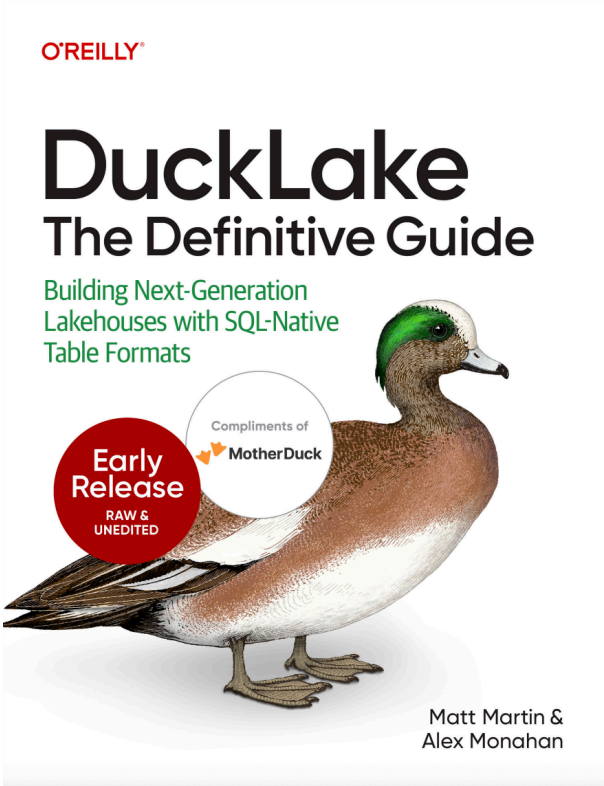
DuckLake unlocks several interesting use cases. Using the *inlining* feature (using the catalog database to stage small updates) allows you to stream data into the data lake. Using its lightweight setup, you only need some storage and a public HTTPS endpoint to serve a read-only lakehouse without authentication. And in the context of DuckDB, it unlocks a multiplayer setup where multiple DuckDB instances can access the same DuckLake while coordinating through a central PostgreSQL catalog database.

Adoption

We are delighted to see how quickly the community adopted DuckLake. The `ducklake` DuckDB extension is now ranked among DuckDB's top-10 core extensions based on the download statistics.

There are now DuckLake clients for Apache DataFusion (initial implementation by Hotdata), Apache Spark (by MotherDuck), Trino (awitten1) (by Andrew Witten), Trino (brikk) (by Jayson Minard), and Pandas dataframe (mostly produced by agentic coding). MotherDuck also offers a hosted DuckLake service, where they manage both the catalog database and the data storage for you. Update: MotherDuck now supports DuckLake v1.0.

We are very proud that DuckLake is already used in production at dozens of companies – see the collection of logos on our new landing page. And there is even an O'Reilly DuckLake book in the making!



For more DuckLake use cases and libraries, see the awesome-ducklake repository.

What's New in DuckLake v1.0

First and foremost, DuckLake ships dozens of bugfixes to make the format robust for production deployments (see the appendix). We are particularly delighted that many contributions were made by community members and we would like to thank them for their work!

In the following, we show some key features of DuckLake v1.0 – available in the DuckDB v1.5.2 that was also released today.

Data Inlining

Data inlining is one of the flagship features of DuckLake. It basically enables performing small insert, delete and update operations in the catalog database, avoiding the proliferation of “the small file problem”. DuckLake v1.0 brings full inlining of updates and deletes. This feature is now on by default with a default threshold of 10 rows. If you want to find out more about this feature, head to this blog we published recently. In this example, we do an insert, delete and update and showcase how this does not create any new files in DuckLake. We then `CHECKPOINT` to flush the inlined data to object storage.

```
CREATE TABLE lake.t (id INT, status VARCHAR);
INSERT INTO lake.t VALUES (1, 'en route'), (2, 'shipped');
DELETE FROM lake.t WHERE id = 1;
UPDATE lake.t SET status = 'delivered' WHERE id = 2;
FROM ducklake_list_files('lake', 't'); -- returns empty
CHECKPOINT; -- flushes data
```

Sorted Tables

If you are often going to run queries against a high cardinality column, like an id or a timestamp, sorting is a great way to increase read query performance. Both row group and file pruning will be benefited for queries that push filters on the sorted keys. Sorted tables support column ordering but also arbitrary SQL expressions. The default approach will sort tables on compaction, flushing or insertion, but the latter one can be disabled to avoid hindering write performance.

```
CREATE TABLE lake.sorted_t (id INT, payload JSON);
ALTER TABLE lake.sorted_t SET SORTED BY (id ASC);
INSERT INTO lake.sorted_t
VALUES
    (33, {'key': 'value'}),
    (2,  {'key': 'value'}),
    (42, {'key': 'value'}),
    (1,  {'key': 'value'});
CHECKPOINT;
FROM lake.sorted_t;
```

Bucket Partitioning

Bucketing works by hashing the value of the target column and using the modulo, based on the number of buckets, to assign this value to a specific bucket. The bucket partitioning functionality uses the murmur3 hash implementation for full compatibility with Iceberg. If you want some of the benefits of partitioning but your column has a high cardinality, bucketing is a good alternative.

```
CALL lake.set_option('data_inlining_row_limit', 0);
CREATE TABLE lake.events (
    user_name VARCHAR, event_type VARCHAR, ts TIMESTAMP);
ALTER TABLE lake.events SET PARTITIONED BY (bucket(8, user_name));
INSERT INTO lake.events VALUES
    ('alice', 'click', '2024-01-01'),
    ('bob', 'view', '2024-01-01'),
    ('charlie', 'click', '2024-01-02');
EXPLAIN ANALYZE FROM lake.events WHERE user_name = 'alice';
```

Type System: Geometry

Following up on the addition of the `GEOMETRY` data type to DuckDB core, better geo stats have been enabled that allow for filter pushdown. `GEOMETRY` types can also now be nested in structs, lists and maps. In the following example, we showcase how a simple filter pushdown would work by using the `&&` operator, which checks overlapping bounding boxes of two geometries.

```
LOAD spatial;

CALL lake.set_option('data_inlining_row_limit', 0);
CREATE TABLE lake.places (name VARCHAR, location GEOMETRY);
INSERT INTO lake.places VALUES ('Amsterdam', ST_Point(4.9, 52.37));
INSERT INTO lake.places VALUES ('London', ST_Point(-0.12, 51.51));
SELECT name
FROM lake.places
WHERE location
  && ST_GeomFromText('POLYGON((4 52, 5 52, 5 53, 4 53, 4 52))');
```

Type System: Variant

Variant is a similar type to JSON but with some very distinctive qualities that make it a preferable alternative, namely: (i) it has support for many more types than JSON, for example `DATE` or `TIMESTAMP`; (ii) it is stored in a binary encoded format rather than a string; (iii) it can be shredded to primitive types, which allows for much better query performance (including filter and projection pushdown). We believe `VARIANT` will eventually replace `JSON` as the main type for semistructured data in database systems.

```
CREATE TABLE lake.events (id INT, payload VARIANT);
INSERT INTO lake.events VALUES
    (1, {'user': 'alice', 'ts': TIMESTAMP '2024-01-01'}),
    (2, {'user': 'bob', 'ts': TIMESTAMP '2024-01-02', 'rand': 'value'});
SELECT *
FROM lake.events
WHERE payload.user = 'bob';
```

Deletion Vectors

Deletion vectors were introduced in the Iceberg v3 specification. While developing DuckLake we are also striving to keep compatibility with Iceberg at the data level. We therefore have implemented deletion vectors stored as Puffin files. These deletion vectors work in a similar way to delete files (which were already a part of the Iceberg specification).

> This feature is experimental and we are planning some improvements for the upcoming DuckLake releases.

```
CREATE TABLE lake.t (id INTEGER);
CALL lake.set_option('write_deletion_vectors', true, table_name => 't');
INSERT INTO lake.t FROM range(100);
DELETE FROM lake.t WHERE id < 5;
```

The Future of DuckLake

Even though today we release DuckLake v1.0, the job is not yet done. We will continue releasing new DuckLake spec versions, and here we outline the current plans for DuckLake v1.1 and possible directions for DuckLake v2.0.

DuckLake v1.1

There are two main features planned for DuckLake v1.1, which will require spec changes:

1. **Variant Inlining.** Currently, variant inlining only works for catalogs that natively support it (e.g., DuckDB). However, if the catalog does not support the Variant type, the table will never be inlined. We will design and implement the necessary specification changes that will allow this.

2. **Multi-Deletion Vector Puffin Files.** DuckLake v1.0 already supports single deletion vector puffin files, but we want to ensure that, similar to our current partial deletion files, DuckLake can store and read multiple deletion vectors in one file, thereby preserving time travel information while still minimizing the small file problem.

DuckLake v2.0

DuckLake v2.0 is most likely not coming anytime soon, as we will focus on maturing our current feature set and guaranteeing the stability of our spec. Although we have a rough sense of the problems we want to tackle, community needs will also strongly drive our plans, so features may be added or reprioritized over time. Below is a list of the features we currently believe are important for DuckLake v2.0:

1. **Git-like branching.** Allowing users to create branches of their DuckLakes and merge changes between them. Similar to what Git is for code, but for data.
2. **Permission-based roles.** Defining roles and permissions to control access to DuckLake objects and operations. This is already possible by configuring Postgres and your S3 bucket correctly, but we want to make this easier to manage directly through DuckLake.
3. **Incremental materialized views.** Supporting materialized views that can be refreshed incrementally by applying tracked changes in DuckLake tables, rather than recomputing the full view.

Conclusion

Today we released DuckLake v1.0, a stable specification for DuckLake, alongside the DuckDB `ducklake` extension compatible with this specification version. We are very happy with the progress that DuckLake has made in just one year, with some great community adoption and a great foundation for future improvements. If you want to hear more from the developers of DuckLake, make sure to check out our podcast episode on DuckLake v1.0:

Appendix

This is all of the work that has gone into DuckLake since the v1.0 target was set; both from the community and the DuckLabs team. In this appendix we are listing 108 PRs that have been merged since the end of 2025. We have classified them so as to convey that the focus has always been making DuckLake stable and production ready, with 68 PRs merged focused on reliability and correctness, 12 focused on internal refactoring to lay the foundation for future developments and another 12 focused on performance improvements.

See the details of the release.